Andrew Long
Prof Jon McGowan
CSIS-2430-501 - Summer
DATE

Programming Project 3

For the third project of our course we are tackling what is known as a knapsack problem. In our example, rather than packing a knapsack we are loading scientific experiments into a space shuttle. The problem that is encountered here is that not all experiments can fit on the shuttle, as we have a limited resource of weight to consider. The assignment provides us with 12 potential experiments, each with a weight value, as well as a rating of value. It is up to our program to assist in providing us the best possible subsets of experiments that can fit onto the shuttle from our main set of 12.

This problem is interesting, as it is deceptively much more complex than one would think at a cursory glance with only 12 items. The issue is that once you start to unpack the problem, you realize that that set of 12 items actually contains 4096 possible subsets that can be examined for potential use. With so few options providing so many potential options, one can see why computing power would be a great fit for finding the optimal solution, especially in similar problems with much larger data sets and potentially multiple parameters.

To start our search for the most optimal subset of experiments we begin by doing what we will refer to as a greedy approach. In our greedy approaches we are going to take an interest in one of the characteristics of our experiment and use that to help determine what it is to bring. Our first greedy approach is focused on the rating of the experiments. To do this, our program is going to take our array of Experiments, a class we created to store the data for each individual experiment, and sort through the array to put the highest rated objects first. We work through the array, adding each experiment along the way, as long as its addition does not put us above our weight limit. What this approach does is give us as many high rated options in our subset as possible. We do a similar thing with our other greedy approaches; one that cares about the lightest weight objects so we can fit the most, and the last with a focus on the experiments with the best ratio of weight to rating. Each of these greedy approaches is accomplished with its own method, along with a helper method to print the results.

These greedy approaches all provide us with potential ways to load our shuttle, but they may not, and likely don't provide us the most optimal way. To find the most optimal way, our program will brute force check all 4096 potential subsets, and keep track of our top three options. To assist with this brute force technique, we are going to use a technique I discovered while researching this problem known as bitmasking (1). Using a loop to search through all 4096 potential options, using a binary bit mask to help in marking what experiments are included in our current iteration of the loop and getting the total rating and total weight of each subset. If the weight for our subset is under our predetermined limit of 700 kg, we continue considering the subset. As our assignment only calls for the best three options, we will compare each subset

under our weight limit to our current top 3 options, and replace each spot as new top options are discovered. This approach gives us a definitive answer to our question, and provides us with the most optimal ways to load our shuttle, at the cost of looking at every possible combination. As mentioned earlier in this report, our limited set of 12 experiments already provides us with 4096, this solution would exponentially rise in scale and needed computational power as the number of items to consider increases.

This leads us to approaching this problem using dynamic programming. This solves our knapsack problem by breaking it into many smaller subproblems and storing the best solution for each one so it does not need to be recalculated. A table is created where the number of rows represent the number of experiments considered and each column represents a possible weight limit from 0 to 700 kgs. For each experiment, the algorithm decides on whether or not to include it based on it producing a higher total rating, without exceeding the weight limit of each specific column. Each table entry stores the best rating found at that specific combination of experiments and weight capacity. Once the table has been filled in, the optimal solution will be left in its final cell, and the included experiments are found by working our way backwards through our table. This approach not only provides us with the same optimal solution as our brute-force does, it accomplishes the task with significantly less problem of scale. Regarding scalability, a brute force approach to this problem has an exponentially increase factor of O(2^*n*), where *n* is the amount of elements within the set; whereas with dynamic programming we have a time complexity of O(*nW*) where *n* is the number of elements and *W* is the maximum weight capacity.

Here are the results of each greedy method's attempts at loading the shuttle.

Greedy Strategy	**Highest Rated**	
Exp	**Weight (kg)**	**Rating**
Solar Flares	264	9
Micrometeorites	170	9
Binary Stars	203	8
Cloud Patterns	36	5
Seed Viability	7	4
Totals	**680**	**35**

Greedy Strategy	**Lightest Weight**	
Exp	**Weight (kg)**	**Rating**
Seed Viability	7	4
Yeast Fermentation	27	4
Cloud Patterns	36	5

Mice Tumors	65	8
Microgravity Plant Growth	75	5
Cosmic Rays	80	7
Sun Spots	90	2
Relativity	104	8
Micrometeorites	170	9
Totals	**654**	**52**

Greedy Strategy	**Best Point to Weight Ratio**	
Exp	**Weight (kg)**	**Rating**
Seed Viability	7	4
Yeast Fermentation	27	4
Cloud Patterns	36	5
Mice Tumors	65	8
Cosmic Rays	80	7
Relativity	104	8
Microgravity Plant Growth	75	5
Micrometeorites	170	9
Sun Spots	90	2
Totals	**654**	**52**

Here is the most optimal strategy of loading the shuttle, found using the brute force method.

Brute Force	**Best Point to Weight Ratio**	
Exp	**Weight (kg)**	**Rating**
Cloud Patterns	36	5
Binary Stars	203	8
Relativity	104	8
Seed Viability	7	4
Mice Tumors	65	8
Micrometeorites	170	9
Cosmic Rays	80	7

Yeast Fermentation	27	4
Totals	**692**	**53**

A comparison of each method.

Method	**Weight (kg)**	**Rating**
Highest Rating	680	35
Lightest Weight	654	52
Best Point to Weight Ratio	654	52
Brute Force	692	53

As you can see from the tables above, our optimal shuttle loading provides us with a rating of 53 and is just 8 pounds under our weight threshold. Our least effective method of loading was our greedy method which prioritized experiments with a higher rating. Our two other greedy methods, came up with a tie for this specific set of experiments, but it found its way to its objects in a different order. I believe that with larger, more varied data set that the greedy method that focused on the experiments with the best point to weight ratio would more often succeed at providing one the closest solution to the knapsack without using brute force or dynamic programming. The reason I believe that that method worked better than the others was it looked at more than one part of the problem. By only looking at one aspect of each experiment we end up ignoring the other and we are put in a place where we are loading our ‘Solar Flares’ on board. At a cursory glance this experiment nets us a 9 rating, but when we look further at said ‘Solar Flares’ we realize we are paying 29.33 kgs per rating, while the ‘Seed Viability’ experiment nets us a point rating for each of its 1.75 kgs.

This project helped in demonstrating how the different approaches to solving this knapsack problem highlight core areas of logic, and computational bandwidth. We see how important it is to consider all factors as we tested the greedy approaches, and how ignoring the weight of each experiment and only focusing on the rating of each left us with a nearly full shuttle, but with a lower rating overall. We saw guaranteed success with our brute force approach, but saw that its exponential computational complexity would make this approach very difficult for even medium sized data sets. Lastly, the dynamic programming part of this project, like our project on the sorting algorithms, helped in my understanding of the importance of algorithmic efficiency and problem optimization.